WPF, XAML, Controls, and Styles

Learning objectives

By the end of these lessons, you should be able to:

- Create a WPF desktop application and explain how XAML and C# code-behind work together.
- Design interfaces using Grid, StackPanel, and Viewbox.
- Configure and manipulate controls through XAML and C#.
- Handle button clicks, checkbox state changes, and radio button selections.
- Use basic data binding to connect interface properties.
- Define reusable WPF styles and apply them implicitly or explicitly.
- Use style triggers to change the appearance of controls dynamically.

Part 1 — Introduction to WPF and XAML

1. Understanding Windows Presentation Foundation

Windows Presentation Foundation (WPF) is a .NET user interface framework for building Windows desktop applications.

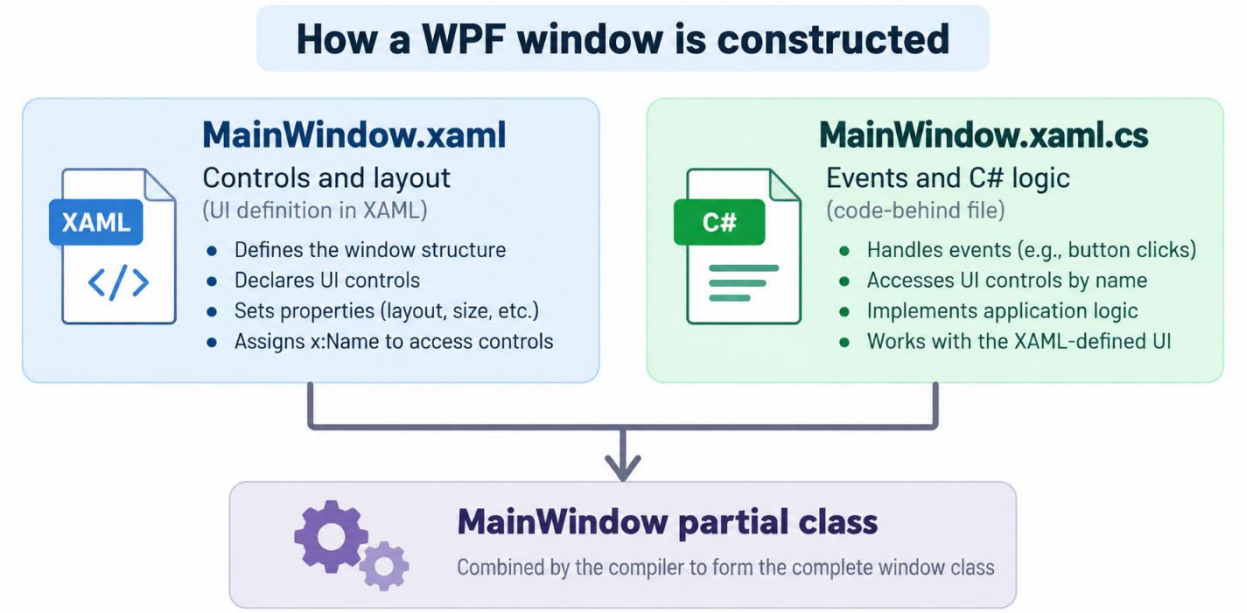
Unlike console applications, WPF applications use graphical windows containing controls such as buttons, text boxes, labels, checkboxes, and radio buttons.

WPF supports several important application development capabilities, including user interface controls, layout management, graphics, resources, data binding, documents, and security.

1.1 XAML and C# code-behind

WPF separates much of the interface definition from the application's procedural logic.

- **XAML (eXtensible Application Markup Language)** describes the controls, layout, and visual properties.
- **C# code-behind** handles events, accesses controls, and implements application behavior.



1.2 Partial classes in WPF

A **partial class** is a class whose definition is distributed across multiple source files.

A typical WPF window's code-behind looks like this:

```
public partial class MainWindow : Window
{
    0 references
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

The partial keyword allows this declaration to be combined with code generated from the XAML definition.

`InitializeComponent()` loads and initializes the window's XAML-defined elements.

Note: The **`.xaml`** file is markup, not C# source code. WPF generates supporting C# code from that markup, which is combined with the code-behind partial class.

2. Lab 1 — Create a WPF Application

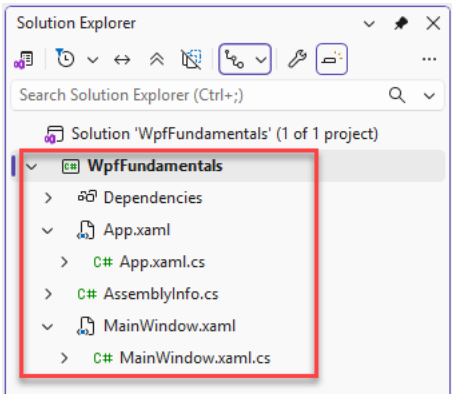
Step 1: Create the project

1. Open Visual Studio.
2. Select **Create a new project**.

3. Search for **`WPF`**.
4. Select **WPF App (.NET Framework)** using C#.
5. Click **Next**.
6. Name the project **`WpfFundamentals`**.
7. Choose an available .NET Framework version and create the project.

Step 2: Examine the project structure

In Solution Explorer, locate:

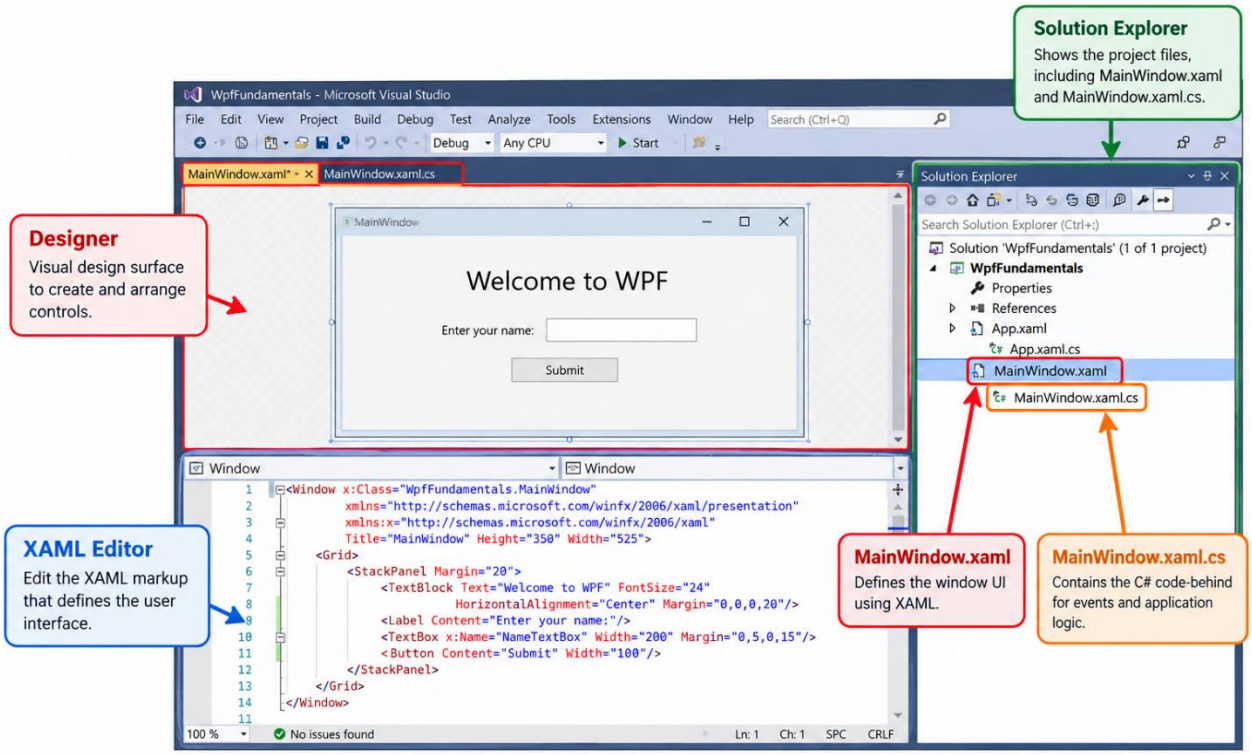


File	Purpose
`App.xaml`	Application configuration and shared resources
`App.xaml.cs`	Application-level C# logic
`MainWindow.xaml`	Main window's interface
`MainWindow.xaml.cs`	Main window's C# code-behind

Step 3: Open the designer

Double-click **`MainWindow.xaml`**.

Visual Studio provides a XAML editor and a graphical designer you can use together.



3. Understanding XAML Syntax

XAML uses XML-style elements to create .NET objects and assign their properties.

A simplified example:

```
<Window>
    <Grid>
        <StackPanel>
            <Label x:Name="NameLabel"
                   Content="Welcome!" />
        </StackPanel>
    </Grid>
</Window>
```

This shows the containment hierarchy:

```
Window
└── Grid
    └── StackPanel
        └── Label
```

The full **`Window`** element in a working project also includes namespace declarations and class information generated by Visual Studio.

3.1 Naming controls

The **`x:Name`** attribute assigns a name that can be used to reference a control in code-behind.

For example:

```
<TextBox x:Name="NameTextBox" />
<Label x:Name="ResultLabel" />
```

The C# code can access them:

```
string name = NameTextBox.Text;
ResultLabel.Content = name;
```

Notice the difference between the properties:

- **`TextBox.Text`** stores the text entered by the user.
- **`Label.Content`** determines what the label displays.

Pitfall: Label does not have a Text property. Use Content to display text in a WPF Label.

4. Basic WPF Controls

Control	Purpose
`Button`	Initiates an action when clicked
`Label`	Displays text or other content
`TextBox`	Accepts user-entered text
`TextBlock`	Displays read-only text, including multiple lines
`CheckBox`	Represents a selectable Boolean or three-state option
`RadioButton`	Represents one choice within a group
`StackPanel`	Arranges child controls sequentially
`Grid`	Organizes controls into rows and columns

`Grid` and **`StackPanel`** are layout containers, not input controls. They determine how other elements are arranged.

5. Handling Button Clicks

WPF applications are event-driven: an event handler executes in response to an action such as clicking a button.

Step 1: Define the button in XAML

```
<Button x:Name="SubmitButton"
        Content="Submit"
        Click="SubmitButton_Click" />
```

The Click attribute identifies the method to execute when the button is clicked.

Step 2: Implement the event handler

In **`MainWindow.xaml.cs`**:

```
private void SubmitButton_Click(
    object sender,
    RoutedEventArgs e)
{
    string name = NameTextBox.Text;
    ResultLabel.Content = $"Hello, {name}!";
}
```

The sender parameter identifies the object raising the event. **`RoutedEventArgs`** contains event information.

Step 3: Test the interaction

Create a text box named **`NameTextBox`** and a label named **`ResultLabel`**, run the application, enter a name, and click Submit.

The label should display the personalized greeting.

6. Introduction to Data Binding

Data binding connects one property to another so WPF can transfer values between them.

6.1 Binding between controls

```
<StackPanel>
    <TextBox x:Name="OutputTextBox" />

    <Label Content="{Binding ElementName=OutputTextBox, Path=Text}" />
</StackPanel>
```

Here:

- **`ElementName=OutputTextBox`** identifies the source control.
- **`Path=Text`** identifies its source property.

- **`Label.Content`** is the destination property.

As the text changes, the label displays the corresponding value without a button-click handler.

6.2 Binding to a code-behind property

In **`MainWindow.xaml.cs`**:

```
public string MyName { get; set; } = "Christine";

public MainWindow()
{
    InitializeComponent();
    DataContext = this;
}
```

In XAML:

```
<Label Content="{Binding Path=MyName}" />
```

`DataContext = this` establishes the window as the binding source.

The binding reads the **`MyName`** property and displays its initial value.

Important limitation: A plain auto-implemented property does not automatically notify the interface when its value changes. This example demonstrates initial property binding, not automatic updates after every subsequent assignment.

Part 2 — WPF Layout Controls

7. Understanding the Grid

A **Grid** is a layout panel that arranges child elements into rows and columns.

By default, a Grid without explicit row or column definitions has one row and one column.

```
<Grid>
    <Ellipse Fill="Red" />
</Grid>
```

The ellipse is placed in the grid's single cell and stretches according to its layout properties.

8. Defining Rows and Columns

8.1 Creating a 2 × 2 Grid

```
<Grid ShowGridLines="True">
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>

    <Grid.RowDefinitions>
        <RowDefinition />
        <RowDefinition />
    </Grid.RowDefinitions>
</Grid>
```

The grid contains two columns and two rows, producing four cells.

`ShowGridLines="True"` displays grid boundaries for debugging and layout inspection.

8.2 Grid sizing options

WPF supports three common sizing approaches.

Sizing	Example	Meaning
Fixed	**`Width="60"`**	Fixed width of 60 device-independent units
Automatic	**`Width="Auto"`**	Size according to content
Proportional	**`Width="*"`**	Receive a proportional share of available space

For example:

```
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="Auto" />
    <ColumnDefinition Width="*" />
    <ColumnDefinition Width="2*" />
</Grid.ColumnDefinitions>
```

The first column is sized according to its content. The remaining available width is divided between the second and third columns in a 1:2 ratio.

For rows, use **`Height`**, not **`Width`**:

```
<RowDefinition Height="Auto" />
<RowDefinition Height="2*" />
```

8.3 Positioning controls

Use **`Grid.Row`** and **`Grid.Column`** to position elements.

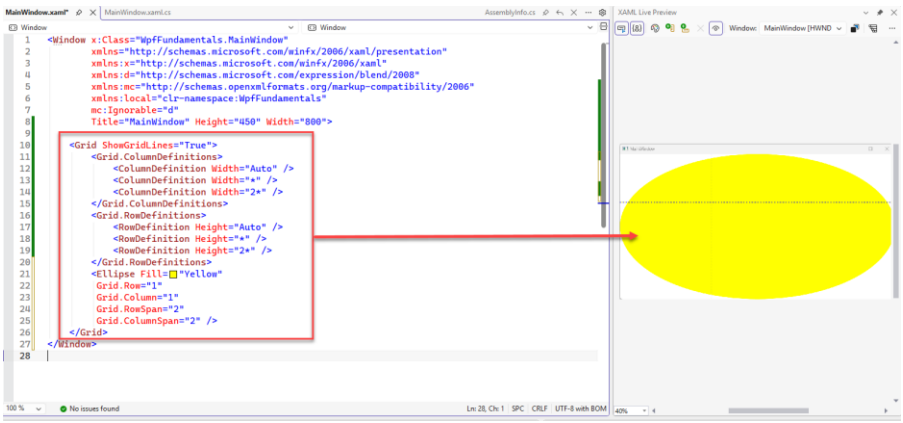
```
<Ellipse Fill="Red"
         Grid.Row="1"
         Grid.Column="2" />
```

Both row and column indices begin at zero.

8.4 Spanning multiple cells

An element can occupy multiple rows or columns.

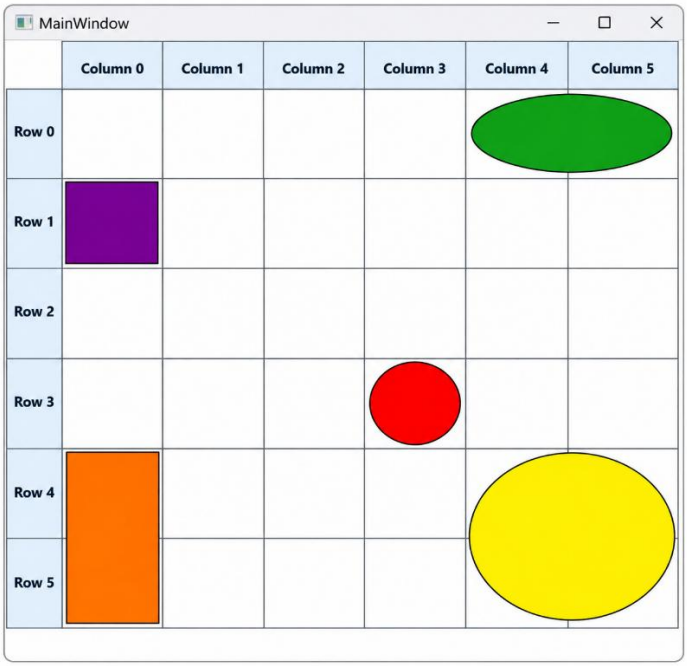
```
<Ellipse Fill="Yellow"
         Grid.Row="1"
         Grid.Column="1"
         Grid.RowSpan="2"
         Grid.ColumnSpan="2" />
```



`Grid.RowSpan` and **`Grid.ColumnSpan`** determine how many cells the element occupies.

9. Lab 2 — Construct a Grid with Shapes

Objective: Reproduce a six-row, six-column layout containing rectangles and ellipses.



Step 1: Define the grid

Replace the existing root Grid with:

```
<Grid ShowGridLines="True">
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>

    <Grid.RowDefinitions>
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
    </Grid.RowDefinitions>

    <!-- Add shapes here -->
</Grid>
```

Step 2: Add the shapes

Replace the comment with:

```
<Rectangle Fill="Purple"
           Grid.Row="1"
           Grid.Column="0" />

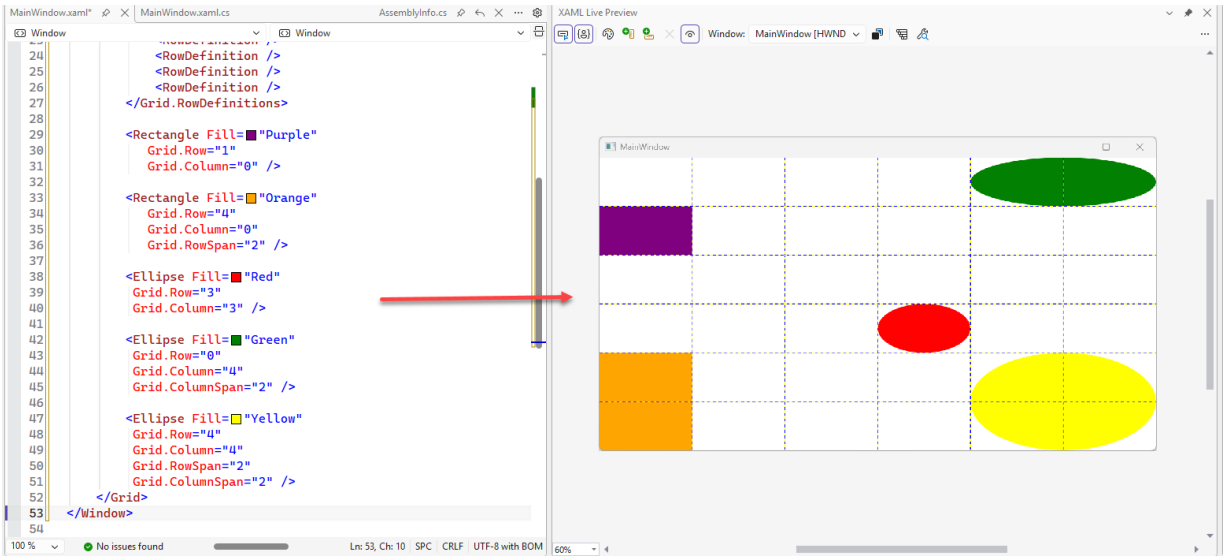
<Rectangle Fill="Orange"
           Grid.Row="4"
           Grid.Column="0"
           Grid.RowSpan="2" />

<Ellipse Fill="Red"
         Grid.Row="3"
         Grid.Column="3" />

<Ellipse Fill="Green"
         Grid.Row="0"
         Grid.Column="4"
         Grid.ColumnSpan="2" />

<Ellipse Fill="Yellow"
         Grid.Row="4"
         Grid.Column="4"
         Grid.RowSpan="2"
         Grid.ColumnSpan="2" />
```

Step 3: Run and inspect



Press **F5** and verify that each shape appears in the intended grid cell.

Resize the window and observe how the shapes and cells respond to the available space.

10. Creating Layout Elements Programmatically

WPF elements can also be constructed in C# rather than declared entirely in XAML.

10.1 Adding a child to a Grid

In XAML:

```
<Grid x:Name="GRD"
      Loaded="GRD_Loaded" />
```

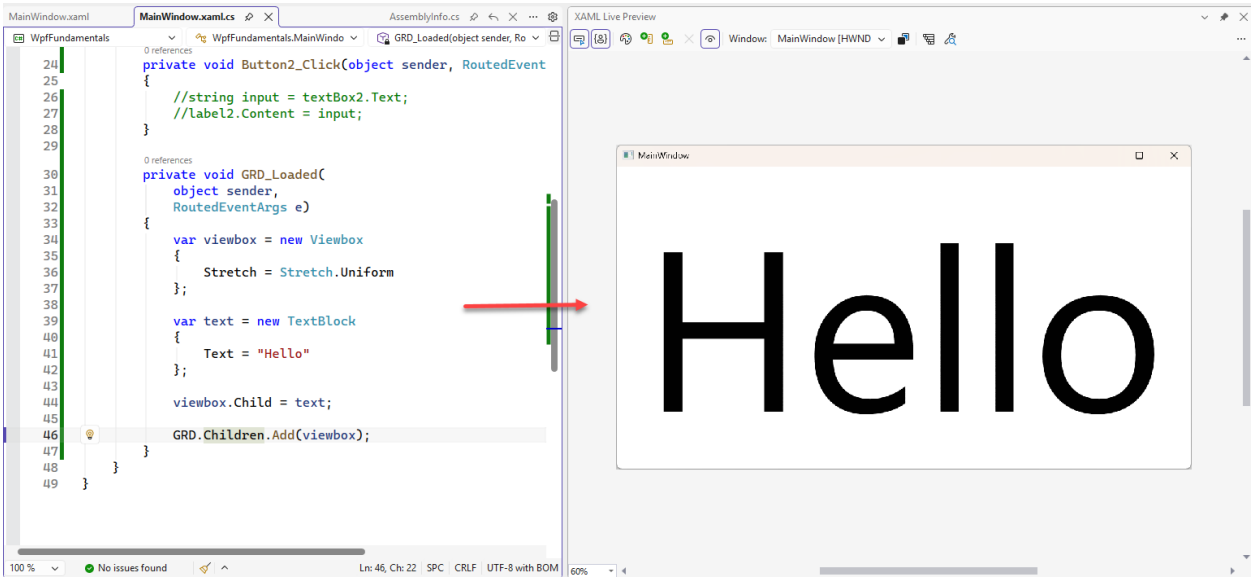
In code-behind:

```
private void GRD_Loaded(object sender, RoutedEventArgs e)
{
    var viewbox = new Viewbox
    {
        Stretch = Stretch.Uniform
    };

    var text = new TextBlock
    {
        Text = "Hello"
    };

    viewbox.Child = text;
    GRD.Children.Add(viewbox);
}
```

The method creates a Viewbox, places a TextBlock inside it, and adds the Viewbox to the grid.



10.2 Adding a child to a StackPanel

XAML:

```
<StackPanel x:Name="STK"
            Loaded="StackPanel_Loaded" />
```

Code-behind:

```
private void StackPanel_Loaded(object sender, RoutedEventArgs e)
{
    var label = new Label
    {
        Content = "Hi"
    };

    STK.Children.Add(label);
}
```

The **`Children.Add()`** method inserts the new control into the layout container.

Part 3 — Viewbox and Scaling

11. Understanding Viewbox

A **`Viewbox`** scales a single child element to fit its available space.

It is useful when text or another visual element should grow or shrink as its container changes size.

Two important properties are:

- **`Stretch`**
- **`StretchDirection`**

11.1 Stretch modes

Mode	Behavior
`None`	No scaling
`Fill`	Fills available space, potentially distorting proportions
`Uniform`	Preserves proportions and fits within available space
`UniformToFill`	Preserves proportions while filling the area, potentially clipping content

11.2 StretchDirection

Value	Behavior
`DownOnly`	Allows shrinking only

`UpOnly`	Allows enlargement only
`Both`	Allows shrinking and enlargement

`StretchDirection` has no effect when **`Stretch="None"`**.

12. Lab 3 — Compare Viewbox Scaling Modes

Create a grid with four columns:

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>

    <Viewbox Grid.Column="0"
             Stretch="None">
        <TextBlock Text="Hello" />
    </Viewbox>

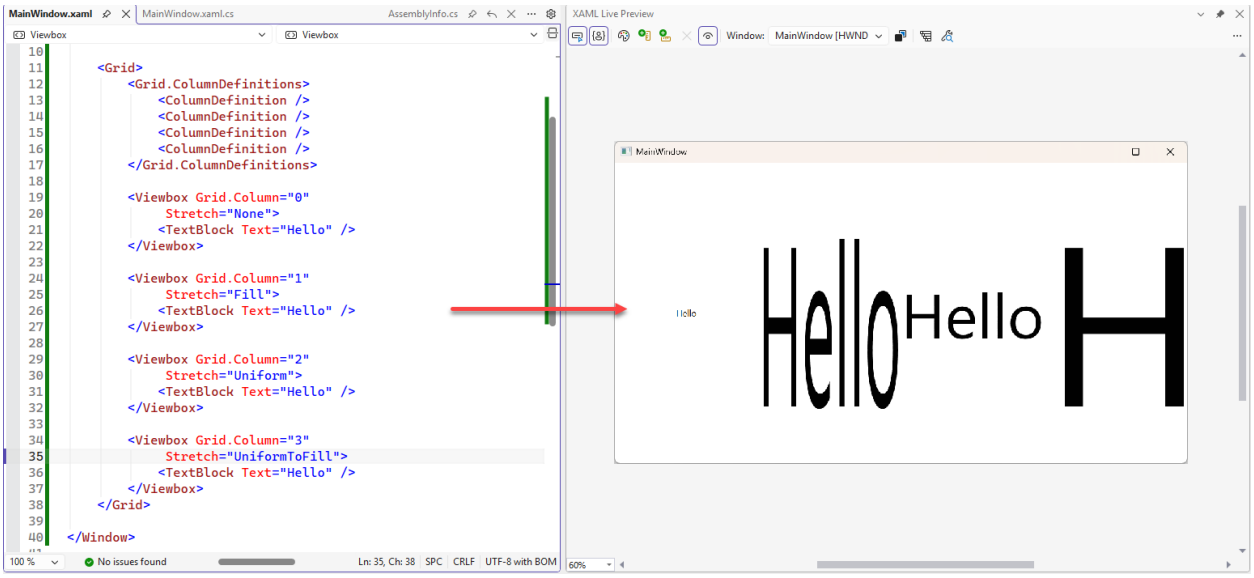
    <Viewbox Grid.Column="1"
             Stretch="Fill">
        <TextBlock Text="Hello" />
    </Viewbox>

    <Viewbox Grid.Column="2"
             Stretch="Uniform">
        <TextBlock Text="Hello" />
    </Viewbox>

    <Viewbox Grid.Column="3"
             Stretch="UniformToFill">
        <TextBlock Text="Hello" />
    </Viewbox>
</Grid>
```

Run the application and resize the window.

Observe which modes preserve the text's proportions, which distort it, and which may cause clipping.




```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>

    <Viewbox Grid.Column="0"
         Stretch="None">
        <TextBlock Text="Hello" />
    </Viewbox>

    <Viewbox Grid.Column="1"
         Stretch="Fill">
        <TextBlock Text="Hello" />
    </Viewbox>

    <Viewbox Grid.Column="2"
         Stretch="Uniform">
        <TextBlock Text="Hello" />
    </Viewbox>

    <Viewbox Grid.Column="3"
         Stretch="UniformToFill">
        <TextBlock Text="Hello" />
    </Viewbox>
</Grid>

</Window>
```

Tip: Add a different background colour to each **`Viewbox`** to make the boundaries easier to identify.

Part 4 — CheckBox and RadioButton Controls

13. Working with CheckBoxes

A **`CheckBox`** represents an option that users can select or clear. WPF checkboxes also support a third, indeterminate state.

13.1 Important CheckBox properties

Property	Description
`IsEnabled`	Determines whether the user can interact with the control
`IsChecked`	Indicates whether the checkbox is checked, unchecked, or indeterminate
`IsThreeState`	Enables the indeterminate state
`ClickMode`	Determines when a mouse interaction activates the control

`IsEnabled="False"` disables the checkbox and typically gives it a greyed-out appearance.

The ClickMode property supports three values:

- **`Release`** — activates when the mouse button is released (default).
- **`Press`** — activates when the mouse button is pressed.

- **`Hover`** — activates when the pointer hovers over the control.

13.2 Three-state checkboxes

When **`IsThreeState="True"`**, the checkbox supports three values:

IsChecked	Meaning
`True`	Checked
`False`	Unchecked
`null`	Indeterminate

For example:

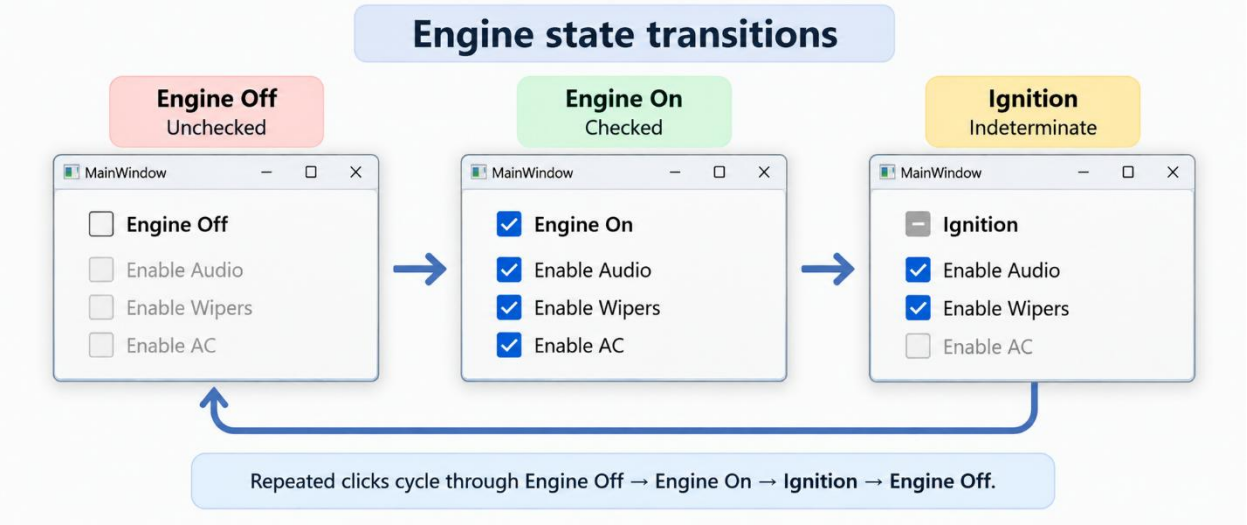
```
<CheckBox x:Name="EngineCheckBox"
          Content="Engine Off"
          IsThreeState="True"
          Checked="EngineCheckBox_Checked"
          Unchecked="EngineCheckBox_Unchecked"
          Indeterminate="EngineCheckBox_Indeterminate" />
```

Each event corresponds to a state transition.

14. Lab 4 — Three-State Engine Control

Objective: Create an interface that uses a three-state checkbox to control three other checkboxes.

The application will represent three engine states.



Step 1 — Create the interface

Replace the existing Grid in **`MainWindow.xaml`** with:

```
<Grid>
    <StackPanel Margin="25">

        <CheckBox x:Name="EngineCheckBox"
                  Content="Engine Off"
                  FontSize="22"
                  IsThreeState="True"
                  IsChecked="False"
                  Checked="EngineCheckBox_Checked"
                  Unchecked="EngineCheckBox_Unchecked"
                  Indeterminate="EngineCheckBox_Indeterminate" />

        <CheckBox x:Name="AudioCheckBox"
                  Content="Enable Audio"
                  IsEnabled="False"
                  Margin="20,20,0,5" />

        <CheckBox x:Name="WipersCheckBox"
                  Content="Enable Wipers"
                  IsEnabled="False"
                  Margin="20,0,0,5" />

        <CheckBox x:Name="AcCheckBox"
                  Content="Enable AC"
                  IsEnabled="False"
                  Margin="20,0,0,5" />

    </StackPanel>
</Grid>
```

Step 2 — Implement the Checked event

Inside **`MainWindow.xaml.cs`**, add:

```
private void EngineCheckBox_Checked(
    object sender, RoutedEventArgs e)
{
    EngineCheckBox.Content = "Engine On";

    AudioCheckBox.IsEnabled = true;
    WipersCheckBox.IsEnabled = true;
    AcCheckBox.IsEnabled = true;

    AudioCheckBox.IsChecked = true;
    WipersCheckBox.IsChecked = true;
    AcCheckBox.IsChecked = true;
}
```

When the main checkbox becomes checked, all three secondary checkboxes are enabled and checked.

Step 3 — Implement the Indeterminate event

```
private void EngineCheckBox_Indeterminate(
    object sender, RoutedEventArgs e)
{
    EngineCheckBox.Content = "Ignition";

    AudioCheckBox.IsEnabled = true;
    WipersCheckBox.IsEnabled = true;

    AudioCheckBox.IsChecked = true;
    WipersCheckBox.IsChecked = true;

    AcCheckBox.IsChecked = false;
    AcCheckBox.IsEnabled = false;
}
```

In the indeterminate state, audio and wipers remain enabled, while air conditioning becomes disabled.

Step 4 — Implement the Unchecked event

```
private void EngineCheckBox_Unchecked(
    object sender, RoutedEventArgs e)
{
    EngineCheckBox.Content = "Engine Off";

    AudioCheckBox.IsChecked = false;
    WipersCheckBox.IsChecked = false;
    AcCheckBox.IsChecked = false;

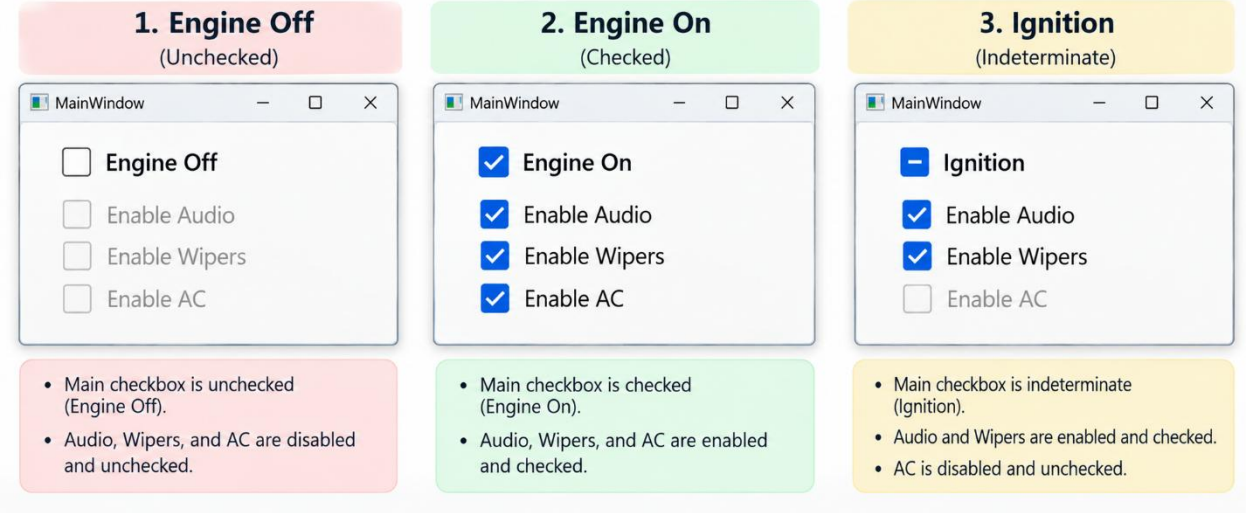
    AudioCheckBox.IsEnabled = false;
    WipersCheckBox.IsEnabled = false;
    AcCheckBox.IsEnabled = false;
}
```

Step 5 — Run and test

Run the application and click the main checkbox repeatedly.

Main state	Audio	Wipers	AC
Engine Off	Disabled, unchecked	Disabled, unchecked	Disabled, unchecked
Engine On	Enabled, checked	Enabled, checked	Enabled, checked
Ignition	Enabled, checked	Enabled, checked	Disabled, unchecked

Pitfall: Events may fire while XAML controls are being initialized. The code above avoids unnecessary initialization events by using **`IsChecked="False"`** as the initial state. If you later change the initial state to True, ensure the dependent controls have been initialized before their properties are accessed.



15. Working with RadioButtons

A **`RadioButton`** represents an option in a group where only one option should be selected at a time.

Unlike checkboxes, which allow independent selections, radio buttons enforce mutual exclusivity within a group.

15.1 Important properties

`IsChecked` indicates whether the radio button is selected.

`GroupName` assigns the radio button to a named group.

```
<StackPanel>
    <RadioButton Content="Small"
                 GroupName="Size" />

    <RadioButton Content="Medium"
                 GroupName="Size" />

    <RadioButton Content="Large"
                 GroupName="Size" />
</StackPanel>
```

Only one radio button in the Size group can be selected at a time.

Radio buttons can also be grouped through their containing layout structure. Using **`GroupName`** makes the intended grouping explicit.

15.2 Creating RadioButtons programmatically

```
var radio = new RadioButton
{
    Content = "Small",
    GroupName = "Size"
};

radio.Checked += Radio_Checked;
radio.Unchecked += Radio_Unchecked;
```

The **+=** operator subscribes an event handler to the specified event.

The corresponding handlers can be declared as:

```
private void Radio_Checked(
    object sender, RoutedEventArgs e)
{
    // Handle selection.
}

private void Radio_Unchecked(
    object sender, RoutedEventArgs e)
{
    // Handle deselection.
}
```

16. Lab 5 — Create Two RadioButton Groups

Objective: Build an interface containing separate foreground and background colour selections.

Step 1 — Define the interface

```
<StackPanel Margin="20">

    <TextBlock Text="Foreground Color:"
               FontSize="18" />

    <RadioButton Content="Red"
                 GroupName="ForegroundGroup"
                 IsChecked="True" />

    <RadioButton Content="Blue"
                 GroupName="ForegroundGroup" />

    <RadioButton Content="Green"
                 GroupName="ForegroundGroup" />

    <TextBlock Text="Background Color:"
               FontSize="18"
               Margin="0,20,0,0" />

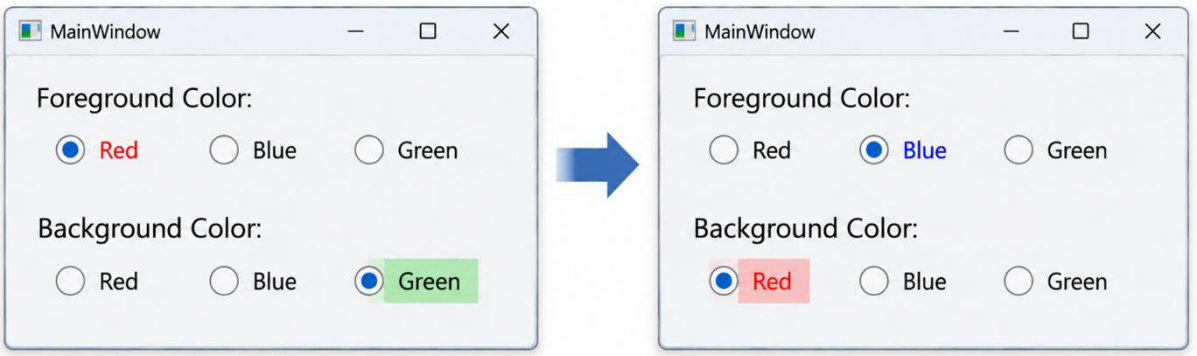
    <RadioButton Content="Red"
                 GroupName="BackgroundGroup" />

    <RadioButton Content="Blue"
                 GroupName="BackgroundGroup" />

    <RadioButton Content="Green"
                 GroupName="BackgroundGroup"
                 IsChecked="True" />

</StackPanel>
```

Radio button groups with selected colours



Initial selection

- Foreground color: Red (text appears in red)
- Background color: Green (radio button has green background)

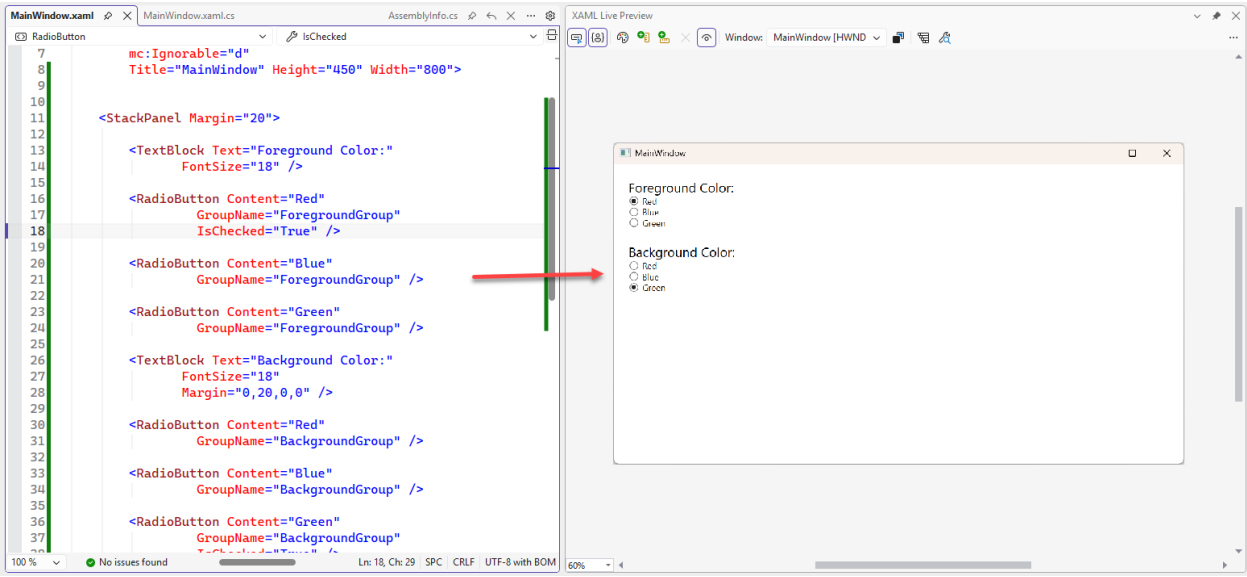
After changing selection

- Foreground color: Blue (text appears in blue)
- Background color: Red (radio button has red background)

Step 2 — Run the application

Verify that:

1. Red is initially selected in the foreground group.
2. Green is initially selected in the background group.
3. Selecting another foreground option deselects the previous foreground option.
4. Foreground selections do not affect background selections.



Step 3 — Modify a colour programmatically

To assign a colour to a radio button's background in code-behind, add a C# event handler:

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
```

```
3 references
private void Radio_Checked(object sender, RoutedEventArgs e)
{
    // Identify which RadioButton was selected
    RadioButton radio = (RadioButton)sender;

    // Change its background to orange
    radio.Background = new SolidColorBrush(Colors.Orange);
}
```

`SolidColorBrush` represents a brush filled with one solid colour.

Note: **`Foreground`** commonly controls the text colour, while **`Background`** specifies the control's background brush. The exact appearance of the radio button's circular indicator depends on its control template.

Part 5 — WPF Styles and Resources

17. Understanding Styles

A WPF **Style** is a reusable collection of property settings that can be applied to one or more interface elements.

Styles help maintain a consistent appearance and reduce repeated XAML declarations.

A style contains **`Setter`** elements, each assigning a value to a property.

```
<Style TargetType="TextBlock">
    <Setter Property="FontSize" Value="14" />
    <Setter Property="FontWeight" Value="Bold" />
    <Setter Property="HorizontalAlignment" Value="Center" />
</Style>
```

The **`TargetType`** specifies which control type the style targets.

18. Applying Styles Implicitly

An implicit style does not have an **`x:Key`**. It applies automatically to matching elements within its resource scope.

For example, place the following inside the root Window element, before its main layout container:

```
<Window.Resources>
    <Style TargetType="TextBlock">
        <Setter Property="HorizontalAlignment"
                Value="Center" />

        <Setter Property="FontFamily"
                Value="Comic Sans MS" />

        <Setter Property="FontSize"
                Value="14" />
    </Style>
</Window.Resources>
```

Now consider:

```
<StackPanel>
    <TextBlock Text="My Pictures" />
    <TextBlock Text="Check out my new pictures!" />
</StackPanel>
```

Both **`TextBlock`** elements receive the same style automatically.

19. Applying Styles Explicitly

An **explicit style** has an **`x:Key`** and is applied by referencing that key.

Step 1 — Define the style

```
<Window.Resources>
    <Style x:Key="TitleText"
           TargetType="TextBlock">

        <Setter Property="HorizontalAlignment"
                Value="Center" />

        <Setter Property="FontFamily"
                Value="Comic Sans MS" />

        <Setter Property="FontSize"
                Value="14" />

    </Style>
</Window.Resources>
```

Step 2 — Apply the style

```
<StackPanel>
    <TextBlock Text="My Pictures"
               Style="{StaticResource TitleText}" />

    <TextBlock Text="Check out my new pictures!" />
</StackPanel>
```

Only the first **`TextBlock`** uses the named style.

`StaticResource` retrieves the resource associated with the specified key.

19.1 Style resource scope

Styles can be declared in different resource collections.

Location	Typical scope
`App.xaml` → **`Application.Resources`**	Application-wide

`MainWindow.xaml` → `Window.Resources`	Window and its descendants
A layout container's Resources	That container and its descendants

For example, an application-wide style can be defined inside **`App.xaml`**:

```
<Application.Resources>
    <Style x:Key="Header1"
           TargetType="TextBlock">

        <Setter Property="FontSize"
                Value="15" />

        <Setter Property="FontWeight"
                Value="ExtraBold" />

    </Style>
</Application.Resources>
```

A window-level style uses the same structure inside **`Window.Resources`**.

19.2 Applying a style programmatically

A named style can also be retrieved from the resource collection in C#:

```
textblock1.Style = (Style)Resources["TitleText"];
```

The resource lookup returns an object, so it must be cast to **`Style`**.

This example assumes **`TitleText`** is accessible through the current window's resources.

Tip: Use implicit styles when all controls of a particular type should share a consistent appearance. Use explicit styles when only selected controls should receive a particular appearance.

20. Style Triggers

A style trigger changes one or more property values when a specified condition becomes true.

For example, a button can change its foreground colour while it is pressed.

```
<Style x:Key="PressedButtonStyle"
       TargetType="Button">

    <Style.Triggers>
        <Trigger Property="IsPressed"
                 Value="True">

            <Setter Property="Foreground"
                    Value="Green" />

        </Trigger>
    </Style.Triggers>

</Style>
```

Apply the style to a button:

```
<Button Content="Submit"
        Style="{StaticResource PressedButtonStyle}" />
```

When **`IsPressed`** becomes **`True`**, the trigger sets **`Foreground`** to green. When the condition is no longer true, the trigger's property value is no longer applied.

21. Lab 6 — Style RadioButtons Based on Selection

Objective: Extend the foreground/background colour interface so that selecting a radio button changes its visual appearance.

Step 1 — Define the styles

Place these styles inside **`Window.Resources`**:

```
<Window.Resources>

    <Style x:Key="ForegroundSelectionStyle"
           TargetType="RadioButton">

        <Style.Triggers>
            <Trigger Property="IsChecked"
                     Value="True">

                <Setter Property="Foreground"
                        Value="{Binding RelativeSource={RelativeSource Self},
                                        Path=Content}" />

            </Trigger>
        </Style.Triggers>

    </Style>

    <Style x:Key="BackgroundSelectionStyle"
           TargetType="RadioButton">

        <Style.Triggers>
            <Trigger Property="IsChecked"
                     Value="True">

                <Setter Property="Background"
                        Value="{Binding RelativeSource={RelativeSource Self},
                                        Path=Content}" />

            </Trigger>
        </Style.Triggers>

    </Style>

</Window.Resources>
```

Step 2 — Apply the styles

For the foreground group:

```
<RadioButton Content="Red"
             GroupName="ForegroundGroup"
             Style="{StaticResource ForegroundSelectionStyle}" />

<RadioButton Content="Blue"
             GroupName="ForegroundGroup"
             Style="{StaticResource ForegroundSelectionStyle}" />

<RadioButton Content="Green"
             GroupName="ForegroundGroup"
             Style="{StaticResource ForegroundSelectionStyle}" />
```

For the background group:

```
<RadioButton Content="Red"
             GroupName="BackgroundGroup"
             Style="{StaticResource BackgroundSelectionStyle}" />
<RadioButton Content="Blue"
             GroupName="BackgroundGroup"
             Style="{StaticResource BackgroundSelectionStyle}" />
<RadioButton Content="Green"
             GroupName="BackgroundGroup"
             Style="{StaticResource BackgroundSelectionStyle}" />
```

Step 3 — Understand the binding

Consider:

```
<Setter Property="Foreground"
        Value="{Binding RelativeSource={RelativeSource Self},
                        Path=Content}" />
```

This means:

- **`RelativeSource Self`** identifies the radio button itself as the binding source.
- **`Path=Content`** retrieves the radio button's **`Content`** value.
- **`Foreground`** receives the corresponding colour value.

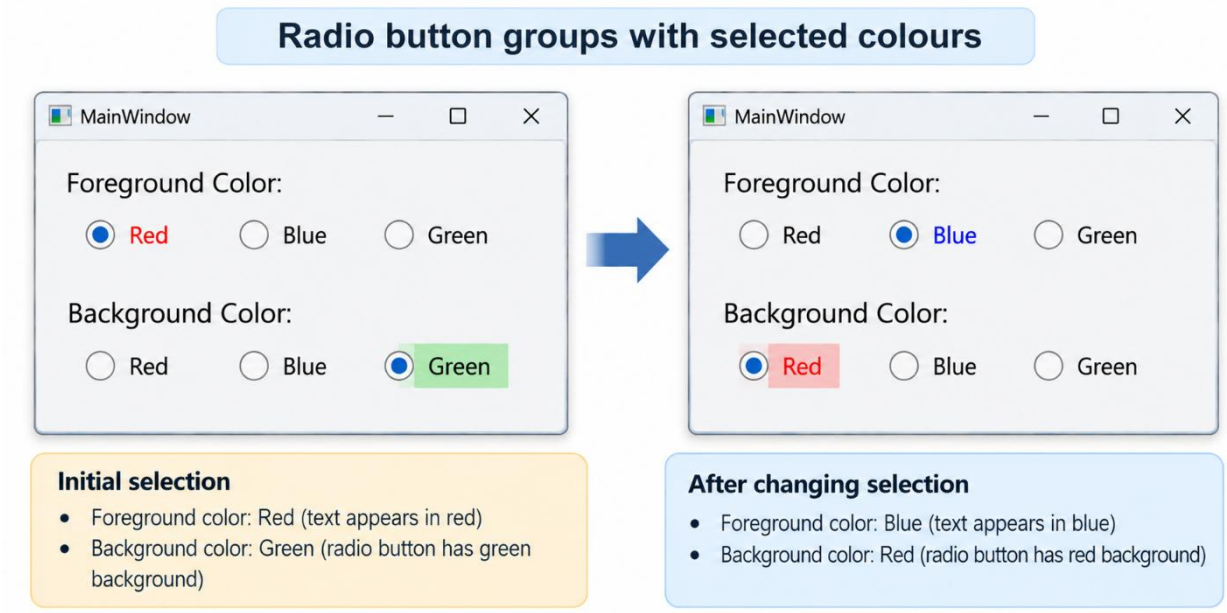
For example, a radio button whose Content is "**`Red`**" uses that value to set its foreground when selected.

Step 4 — Test the application

Run the application and select different radio buttons.

Confirm that the selected foreground option changes its text colour and that the background style changes the control's background brush.

Important: Changing a **`RadioButton`**'s **`Background`** property does not necessarily recolour its circular selection indicator. That appearance depends on the control template. This exercise demonstrates property styling and triggers, not custom control templates.



Part 6 — Consolidated Practical Exercise

22. Build a WPF Control Demonstration Application

Create a WPF application named **`WpfControlsDemo`** that demonstrates the main concepts covered in these lessons.

The application should contain:

- A **`Grid`** with multiple rows and columns.
- A **`StackPanel`** containing text input, a label, and a button.
- A button-click handler that reads the text box and updates the label.
- A **`Viewbox`** that scales a **`TextBlock`**.
- A **`three-state`** checkbox controlling three secondary checkboxes.
- Two independent radio button groups.
- At least one implicit style and one explicit style.
- A style trigger that changes a control property when its state changes.

Recommended implementation sequence

1. **Create the project and layout.** Define the main **`Grid`**, establish rows and columns, and use a **`StackPanel`** to organize related controls.

2. **Add basic interaction.** Implement a button event handler that reads from a **`TextBox`** and updates a **`Label`**.
3. **Add dynamic controls.** Implement the three-state checkbox and independent radio button groups.
4. **Apply styles.** Define a shared implicit style, an explicitly named style, and a trigger for selected controls.
5. **Test the interface.** Verify event handling, state transitions, resource references, control grouping, and layout behavior when resizing the window.

23. Common WPF Errors and Troubleshooting

Problem	Likely cause	Corrective action
A label does not display updated text	Using Text instead of Content	Use **`Label.Content`**
A named control cannot be accessed from C#	Missing or incorrect **`x:Name`**	Verify the control name
A button click does nothing	Missing or incorrectly named event handler	Check the **`Click`** attribute and handler signature
Controls appear in the wrong Grid cell	Incorrect row or column index	Check zero-based indices
A resource cannot be found	Incorrect key or resource scope	Check **`x:Key`** and resource location
Radio buttons affect the wrong group	Incorrect or shared **`GroupName`**	Assign distinct group names
A checkbox handler causes a null-reference error	Event fires before dependent controls are initialized	Review initialization order
A bound property does not update after a C# assignment	No change notification mechanism	Recognize the limitation of basic property binding

24. Key Concepts to Remember

WPF applications combine declarative interface design with event-driven C# programming.

The most important distinctions are:

Concept	Key idea
XAML	Declares interface elements and properties
Code-behind	Implements C# behavior and event handlers
Partial class	Combines the window's generated and manually written C# definitions
Grid	Organizes controls in rows and columns
StackPanel	Arranges controls sequentially
Viewbox	Scales a child element
CheckBox	Supports independent selection and optionally three states
RadioButton	Supports mutually exclusive selection within a group
Data binding	Connects properties
Style	Reuses sets of property values
Style trigger	Changes property values in response to a condition

Expected outcome: You should be able to build, run, and troubleshoot a small WPF desktop application using XAML, C# code-behind, layout containers, interactive controls, data binding, reusable styles, and triggers.